



COMAL Queue Technical Documentation

Version 1.0

Comal++ / Troyanos Engineering

14 September 2026

Software baseline: 0.2.0

Source baseline: bf71e73927fc7a24ec01501c38122327a64ffa4f

Contents

1	Product documentation	4
1.1	Product vision	4
1.2	Business context	5
1.3	Scope	6
1.4	Glossary	7
2	Users and research	9
2.1	User groups	9
2.2	Role-based personas	10
2.3	User scenarios	11
2.4	User journeys	12
3	Requirements	14
3.1	Product requirements baseline	14
3.2	Functional requirements	15
3.3	Non-functional requirements	16
3.4	User stories	17
3.5	Acceptance criteria	19
3.6	Requirements traceability	20
4	User experience	22
4.1	Interface overview	22
4.2	Cashier interface	22
4.3	Client/public display	24
4.4	Administration interface	25
4.5	UX guidelines	25
5	Architecture	27
5.1	System overview	27
5.2	Frontend architecture	28
5.3	Backend architecture	29
5.4	Data flow	30
5.5	Database	31
5.6	Turn and queue domain	33
5.7	Realtime communication	34
5.8	Multimedia architecture	36
5.9	Authentication and authorization	37

5.10	Observed architectural decisions	38
6	HTTP API	40
6.1	API overview	40
6.2	Endpoints	41
6.3	Data contracts	43
6.4	Error handling	44
7	Quality	47
7.1	Quality strategy	47
7.2	Testing strategy	48
7.3	Critical test cases	49
7.4	Known limitations	51
7.5	Technical debt register	52
8	Development process	54
8.1	Development setup	54
8.2	Project structure	56
8.3	Coding standards	57
8.4	Git workflow	58
8.5	Configuration	59
8.6	Environment variables	60
9	Operations	62
9.1	Deployment	62
9.2	Production considerations	63
9.3	Monitoring	64
9.4	Backup and recovery	64
9.5	Security	65
10	Planning	67
10.1	Current engineering status	67
10.2	Product roadmap	68
10.3	Technical roadmap	69
10.4	Release plan	70
10.5	Project metrics	71
11	Reference	73
11.1	Terminology and naming rules	73

11.2 Repository map	74
11.3 Documentation status and confidence	75

List of Code Listings

1 Turn response contract	43
2 Announcement event contract	43
3 Multimedia and settings contracts	44
4 Playlist catalog and application-state contracts	44

1 Product documentation

1.1 Product vision

Confirmed product statement. COMAL Queue is a local application for publishing ticket numbers after orders are ready for pickup. A cashier copies the number from an existing ticket, optionally assigns pickup counter 1 or 2, and the system immediately stores and broadcasts the ready turn. A public display shows the ready list and a time-limited visual callout.

The source and interface identify the operating context as the Troyanos/Comal++ cafeteria associated with the Facultad de Informática, Universidad Autónoma de Querétaro. The exact product name is inconsistent: repository/package text uses COMAL, Comal++, and comal-local, while the latest interface is branded Troyanos. The commercial or institutional naming decision is Requires stakeholder validation.

Problem addressed. The application bridges the period between preparation outside this system and customer pickup. It provides a shared, persistent view of which printed ticket numbers are ready and lets staff remove them after delivery or an erroneous capture.

Operational boundary. This is not a point-of-sale, preparation, kitchen, payment, inventory, or ticket-printing system. Its first implemented state is already ready; all earlier order activity occurs elsewhere.

Product principles evidenced by the repository:

- queue operation and bundled visual/audio resources are local;
- SQLite preserves operational state across process restarts;
- Socket.IO distributes changes to concurrent browser views;
- YouTube is optional and explicitly fails back to local visuals;
- repeated network submissions are made idempotent by caller-supplied request identifiers.

Evidence: README.md; package.json; src/pages/CashierPage.tsx; src/pages/PublicPage.tsx; server/state.service.ts

1.2 Business context

Confirmed context

UI labels, image alternatives, package metadata, and the previous README consistently place the application in a university cafeteria setting at UAQ's Facultad de Informática. Staff already receive a printed two-digit ticket identifier from another process. COMAL Queue begins only when that order is ready.

The operational value implemented in code is narrow: eliminate a transient verbal-only ready list, keep the list visible on a public screen, and support an explicit repeat call. The application does not contain monetary, inventory, menu, customer, or order-line data.

Documented intent requiring validation

The previous README described one cashier station, three televisions showing the same content, available speakers, and a preference for mirrored HDMI or wired local networking. These are repository statements, not enforced deployment configuration. Hardware, cabling, screen count, accessibility expectations, data-retention policy, operating hours, ownership, and support responsibilities remain Requires stakeholder validation.

The previous README also stated that core operation should work without Internet. Code supports local queue entry, persistence, synchronization, local audio assets, and local music without Internet; YouTube is the sole deliberate remote dependency. Public-display audio activation is incomplete, so the offline-audio objective is only partially achieved.

Success definition

No business KPIs, service-level objective, queue-time metric, throughput target, or adoption target is documented. A minimum stakeholder-approved success definition is still required.

Evidence: README.md at audit baseline; index.html; package.json; public/; data/music/; src/components/MediaPlayer.tsx

1.3 Scope

Current scope

- IMPLEMENTED: manually create ready turns numbered 01–99, with optional counter 1 or 2.
- IMPLEMENTED: reject duplicate active numbers and safely replay identical create/announce requests.
- IMPLEMENTED: reorder a recalled turn, edit its counter, mark it delivered, or cancel it.
- IMPLEMENTED: persist turns, request records, multimedia selection, and display settings in local SQLite.
- IMPLEMENTED: synchronize state and new announcement events through Socket.IO.
- IMPLEMENTED: show up to five turns per public-display page, automatically or manually rotate pages, and display a visual announcement.
- IMPLEMENTED: catalog and serve local music; configure fallback, local, or YouTube media.
- IMPLEMENTED: build a Windows x64 NSIS Electron package.

Partial scope

- PARTIALLY_IMPLEMENTED: public audio. Assets, sequencing, Web Audio playback, media ducking, and a cashier-side test exist, but the public display never calls the exposed audio-enablement function.
- PARTIALLY_IMPLEMENTED: multi-screen deployment. The server can bind to a configured host, but the packaged Electron launcher forces loopback. The historical three-screen configuration is not codified.
- PARTIALLY_IMPLEMENTED: administration. Media and display settings exist inside the cashier workspace, with no distinct administrator identity or permission boundary.
- PARTIALLY_IMPLEMENTED: history. The API returns the newest 200 records, but no history screen, filtering, export, retention, or purge exists.
- PARTIALLY_IMPLEMENTED: responsive and accessible presentation. Semantic labels, focus styling, a skip link, and reduced-motion CSS exist, but no automated accessibility or browser matrix is present.

Planned or evidenced scope

The old README calls for a defined backup/restore procedure and reliable offline voice announcements. Those are PLANNED_OR_REFERENCED; no backup automation exists and public audio activation is incomplete. The repository contains unused mascot and reference image artifacts, but their mere presence is not treated as a committed feature.

Explicitly documented out of scope

The previous README explicitly excludes preparation tracking, kitchen workflow, POS integration, charges, refunds, inventory, invoicing, item management, and ticket printing. Current source contains none of these capabilities. Cloud hosting and Internet access are not required for the core queue, although YouTube is an optional connected feature.

Evidence: `server/state.service.ts`; `server/database.ts`; `server/events.gateway.ts`; `src/`; `electron/main.cjs`; `README.md` at audit baseline

1.4 Glossary

Term	Repository-grounded meaning
COMAL Queue	Canonical documentation name for the repository. The UI also uses Troyanos and historical text uses Comal++.
Turn / Turno	One stored ready-order record with an internal UUID and visible two-digit ticket number.
Ticket number	Existing code copied by staff, normalized by the UI to 01–99; it is not the database identity.
Ready / Listo	The only active turn status. A newly created turn enters this state immediately.
Delivered / Entregado	Closed status selected after pickup; the turn remains in history but leaves the active snapshot.
Cancelled / Retirado	Closed status used for an erroneous capture; it remains in history and leaves the display.
Recall / Reannounce	Explicitly raise a ready turn's rank, update its last-announced timestamp, and emit a new announcement.
Counter / Mostrador	Pickup location encoded as 0 (none), 1, or 2.
Cashier workspace	Operational interface rendered on all SPA routes except those beginning with <code>/pantalla</code> .

Public display	Read-only screen at /pantalla; it renders the queue, media, footer, and visual announcement overlay.
Announcement	Ephemeral Socket.IO payload causing a visual callout and, when enabled, sequential turn/counter audio. It is not stored as its own table.
Snapshot	Current ready turns plus multimedia, playlists, and settings emitted to clients.
Rank	Monotonically increasing integer defining newest/recalled-first ordering across turn history.
Request ID	Client-generated idempotency key stored with a fingerprint and serialized response.
Fallback media	The non-YouTube visual carousel shown when YouTube is inactive, unavailable, blocked, or not selected.
Playlist	Server-discovered local directory of supported audio files, or a YouTube playlist identifier when media type is YouTube.

2 Users and research

2.1 User groups

No repository user research, interview notes, analytics, or usability study exists. The groups below are operational roles inferred from implemented interfaces and documentation; they are not authenticated account roles.

Cashier or queue operator — confirmed

Goals and responsibilities: copy a ready ticket number, optionally select a counter, keep the ready list accurate, repeat a call, close a delivered turn, cancel an erroneous capture, and configure display/media behavior.

Permissions in practice: unrestricted access to every HTTP mutation because authentication is absent. The main screen labels the session “Estación de caja.”

Primary interface: CashierPage, rendered for `/`, `/caja`, `/admin`, `/display`, and any other non-`/pantalla` frontend path once the SPA is served.

Customer or public-display viewer — confirmed

Goal: recognize that a ticket is ready and where to collect it. The public interface is read-only at the component level, shows five turns per page, and visually emphasizes the newest or currently announced turn.

Permissions in practice: the UI contains no mutation controls, but the back-end does not enforce a public role. Anyone able to reach the HTTP API can mutate state.

Maintainer or installer — inferred

The repository includes developer scripts, audio generators, local-media directory rules, production build commands, and Electron packaging. A technical maintainer is therefore needed to install, configure, back up, and troubleshoot the system. Responsibilities and operational ownership are not documented.

Administrator — not implemented as a role

Media and settings are administrative capabilities, but no administrator identity, login, separate page component, route guard, or permission model exists. Treating “administrator” as a security role would be inaccurate.

Evidence: `src/pages/CashierPage.tsx`; `src/pages/PublicPage.tsx`;
`src/components/ReadyList.tsx`; `server/app.controller.ts`

2.2 Role-based personas

These personas deliberately omit demographics and fictional biographies.

P-OPS-001 — Queue operator

Role: staff member at the cashier station. **Primary tasks:** enter a ticket after preparation, correct its pickup counter, reannounce it, and close it after pickup or error. **Interaction pattern:** brief, repeated, time-sensitive actions; the form keeps focus on the ticket input after successful creation. **System dependency:** local server and Socket.IO connection; mutation controls are disabled while disconnected. **Unvalidated assumptions:** training level, language needs beyond the Spanish UI, keyboard/scanner use, and error frequency.

P-DISPLAY-001 — Public viewer

Role: person waiting for an already purchased order. **Primary task:** match a printed ticket with a large displayed number and optional counter. **Interaction pattern:** passive viewing; may see automatic page rotation, a visual announcement overlay, media, date/time, and footer messages. **System dependency:** a live local Socket.IO connection for current data. Cached React state remains visible after disconnection, with a stale-data warning. **Unvalidated assumptions:** viewing distance, display size, ambient noise, color/vision needs, and acceptable rotation interval.

P-MAINT-001 — Technical maintainer

Role: person responsible for installation and recovery. **Primary tasks:** install Node dependencies or a desktop build, configure host/port/data paths, manage music files, inspect health, run tests, and back up SQLite safely.

Unvalidated assumptions: who fills this role, access rights, response time, and available tools.

2.3 User scenarios

SC-QUEUE-001 – Add a ready turn

The operator enters one or two digits and selects no counter, counter 1, or counter 2. The frontend validates the range, pads a single digit, and submits an idempotency key. The server validates again, rejects an active duplicate, inserts a ready turn and request record in one transaction, emits state, then emits an announcement. The cashier and public views update.

SC-QUEUE-002 – Repeat a call

The operator selects an active turn and requests reannouncement. The server verifies the turn is ready, raises its rank, updates `last_announced_at`, records the request, and emits state plus a fresh announcement. No new turn is created.

SC-QUEUE-003 – Close a turn

For successful pickup, the operator marks the turn delivered. For an erroneous capture, the operator confirms removal and the server marks it cancelled. Either transition removes the turn from snapshots/public display but retains it in the turns table and history endpoint.

SC-DISPLAY-001 – Observe a disconnection

When Socket.IO disconnects, the last in-memory state remains rendered. The cashier sees mutations disabled; the public screen warns that information may be stale. On reconnect, the server emits a fresh snapshot but does not replay historical announcement events.

SC-MEDIA-001 – Select media

The operator can select a discovered local playlist or supply a validated HTTPS YouTube video/playlist URL. Configuration is persisted and broadcast. Local queue operation remains available if YouTube is offline or fails. In the

current frontend, public media audio stays muted because audio enablement is never invoked.

SC-SET-001 – Change display settings

The operator changes announcement duration or footer messages, or toggles automatic page rotation in the top bar. Server-side validation constrains duration and messages; the updated snapshot reaches all clients.

Evidence: `src/pages/CashierPage.tsx`; `src/components/ReadyList.tsx`;
`src/components/MultimediaPanel.tsx`; `src/hooks/useSystem.tsx`;
`server/state.service.ts`

2.4 User journeys

Ready-to-pickup journey

```
Preparation outside COMAL Queue
|
Operator enters existing ticket and optional counter
|
Server validates + stores READY turn + request record
|
Socket.IO state and announcement events
|
Public display shows turn and visual announcement
|
Customer presents ticket and collects order
|
Operator marks DELIVERED; turn leaves active display
```

Capture-error journey

```
READY turn selected in cashier list
|
Operator chooses "Retirar por error de captura"
|
Confirmation dialog
|
PATCH status=CANCELLED
|
```

Turn remains in history but leaves active display

Recall journey

Operator selects READY turn -> POST announce

- > rank and last-announced time change
- > new ephemeral announcement emitted
- > turn becomes first in ready order

Local-media journey

Maintainer copies supported audio into data/music/<genre>

- > operator refreshes catalog
- > server resolves and allow-lists files
- > operator selects playlist
- > public client requests /media/<genre>/<filename>
- > tracks advance on ended events

The intended audio step in the pickup journey is not included as achieved behavior: assets and playback code exist, but the public screen cannot set audioEnabled to true.

3 Requirements

3.1 Product requirements baseline

This baseline reconstructs requirements from code, tests, UI copy, and the previous README. It is descriptive, not a substitute for owner approval.

ID	Product requirement	Evidence/status
PR-001	The product manages only orders already ready for pickup, using an existing printed ticket number.	Source and prior README; IMPLEMENTED.
PR-002	Core queue entry, display, persistence, and synchronization operate on local infrastructure without an Internet dependency.	Source; IMPLEMENTED.
PR-003	A connected public display presents ready ticket numbers and pickup counter information.	Source/tests; IMPLEMENTED.
PR-004	A new or recalled turn produces a visual and audible call without duplicate calls from retried requests.	Visual/idempotency implemented; public audio PARTIALLY_IMPLEMENTED.
PR-005	Optional connected media must not prevent the local queue from operating.	YouTube fall-back code; IMPLEMENTEDfor queue independence.
PR-006	Closed turns stop appearing but remain persistently recorded.	SQLite, service, and tests; IMPLEMENTED.
PR-007	Operational changes are restricted to authorized staff.	Security expectation inferred from interface; NOT_IMPLEMENTEDand requires stakeholder confirmation.
PR-008	Data retention, backup, recovery, and production support are defined.	Prior README only; PLANNED_OR_REFERENCED.

Undocumented product decisions: authoritative product name, owner, release definition, supported topology, retention period, screen/audio hardware,

operator authentication need, measurable service targets, and legal/licensing constraints.

3.2 Functional requirements

ID	Requirement	Status	Evidence
FR-QUEUE-001	Accept a manually entered ticket from 01 through 99 and create it directly as ready.	IMPLEMENTED	CashierPage.tsx; state.service.ts
FR-QUEUE-002	Reject a second ready turn with the same visible number.	IMPLEMENTED	partial unique index and service conflict
FR-QUEUE-003	Allow counter 0, 1, or 2 on creation and on an active turn.	IMPLEMENTED	validation, UI, service
FR-QUEUE-004	Reannounce only ready turns, raise their display rank, and emit a new call.	IMPLEMENTED	announce() and tests
FR-QUEUE-005	Close a ready turn as delivered or cancelled and exclude it from active snapshots.	IMPLEMENTED	update() and tests
FR-QUEUE-006	Permit reuse of a number after its prior turn closes, with a new internal ID.	IMPLEMENTED	tests and unique-index predicate
FR-QUEUE-007	Return up to 200 historical turns ordered by rank descending.	IMPLEMENTED	history()
FR-QUEUE-008	Prevent repeated create/announce requests with the same ID from duplicating their mutation or announcement.	IMPLEMENTED	requests table and tests
FR-DISPLAY-001	Show ready turns newest/recalled first, five per page, with counter or pickup text.	IMPLEMENTED	PublicPage.tsx
FR-DISPLAY-002	Auto-rotate pages every eight seconds unless disabled; allow manual next-page action when disabled.	IMPLEMENTED	PublicPage.tsx
FR-DISPLAY-003	Show each live announcement for at least the configured duration and serialize queued announcements.	Visual: IMPLEMENTED	announcement hook
FR-AUDIO-001	Play a bundled turn clip followed by a counter/ready suffix and duck media during the call.	PARTIALLY_IMPLEMENTED	playback exists; public activation is unreachable

FR-MEDIA-001	Discover allow-listed local audio files by genre and serve only catalogued files.	IMPLEMENTED	MediaService; tests
FR-MEDIA-002	Persist fallback/local/YouTube selection, playing state, volume, and mute.	IMPLEMENTED	validation, service, and database
FR-MEDIA-003	Accept only HTTPS YouTube URLs for supported hosts and valid video or playlist IDs on the server.	IMPLEMENTED	server URL validation
FR-MEDIA-004	Show local carousel/notice when YouTube is offline, blocked, invalid, or unavailable.	IMPLEMENTED	MediaPlayer .tsx
FR-SET-001	Persist announcement duration, page rotation, and one to ten footer messages.	IMPLEMENTED	settings validation/service
FR-SYNC-001	Emit a complete state snapshot on connection and after mutation; emit announcements only for new/recall actions.	IMPLEMENTED	gateway and integration test
FR-AUTH-001	Authenticate operators and authorize mutations.	NOT_IMPLEMENTED	no auth dependency, guard, user, token, or session

No requirement exists for deleting historical database rows; UI “removal” maps to cancelled, not SQL deletion.

3.3 Non-functional requirements

Confirmed characteristics

ID	Characteristic	Evidence
NFR-LOC-001	Core application assets, queue API, database, WebSocket synchronization, local music, and announcement WAVs are locally hosted.	build/source/assets
NFR-DATA-001	SQLite uses WAL, foreign-key enforcement, and a 5-second busy timeout; mutations use BEGIN IMMEDIATE.	database.ts
NFR-INT-001	Active-number uniqueness is enforced in both service logic and a partial unique index.	database.ts; tests

NFR-REL-001	Create/announce operations are idempotent for a valid request ID and fingerprint.	<code>state.service.ts</code> ; tests
NFR-NET-001	Client HTTP calls time out after eight seconds; Socket.IO reconnects with an 800 ms to 5 s delay.	frontend services/hooks
NFR-UX-001	Styles include breakpoints at 1180, 1000, 760/700, and 480 px, visible focus treatments, a skip link, semantic labels, live regions, and reduced-motion rules.	frontend/CSS
NFR-PORT-001	The package requires Node 22.13+ and declares a Windows x64 NSIS target.	<code>package.json</code>
NFR-SEC-001	Request bodies reject unknown fields; ticket, counter, settings, media, and YouTube inputs are allow-listed and range-checked.	<code>validation.ts</code>

Desired but undefined requirements

Availability, acceptable state propagation latency, maximum concurrent screens, history retention, backup recovery point/time, supported browsers, minimum display resolution and viewing distance, accessibility conformance, performance budgets, maximum queue length, media-storage capacity, log retention, security threat model, and maintenance response times all **require stakeholder definition**. No numerical target is asserted.

The previous README requires offline core operation, but public audio activation currently prevents classifying offline audible announcements as satisfied.

3.4 User stories

US-QUEUE-001 — Record a ready ticket

Status: IMPLEMENTED. As a queue operator, I want to copy the existing ticket number when an order is ready so that waiting customers can see it. Related: FR-QUEUE-001/002/003.

US-QUEUE-002 — Repeat a call

Status: IMPLEMENTED. As a queue operator, I want to reannounce a ready turn without duplicating it so that a customer gets another prompt. Related:

FR-QUEUE-004/008.

US-QUEUE-003 — Complete pickup

Status: IMPLEMENTED. As a queue operator, I want to mark a turn delivered so that it leaves the public ready list while remaining in history. Related: FR-QUEUE-005/006.

US-QUEUE-004 — Correct an erroneous capture

Status: IMPLEMENTED. As a queue operator, I want to confirm removal of a wrongly entered turn so that it no longer appears. Related: FR-QUEUE-005.

US-DISPLAY-001 — Identify a ready order

Status: IMPLEMENTEDvisually. As a waiting customer, I want to see large ready numbers and their pickup counters so that I can recognize my order. Related: FR-DISPLAY-001.

US-AUDIO-001 — Hear a call

Status: PARTIALLY_IMPLEMENTED. As a waiting customer, I want to hear my number and pickup instruction so that I do not need to watch continuously. Playback code and assets exist, but the public display cannot enable them. Related: FR-AUDIO-001.

US-MEDIA-001 — Use local waiting media

Status: IMPLEMENTEDwith muted-audio limitation. As an operator, I want to select a local playlist so that the display has offline media. Related: FR-MEDIA-001/002.

US-MEDIA-002 — Use optional YouTube media

Status: IMPLEMENTEDwith environmental dependency. As an operator, I want to configure a YouTube video or playlist so that connected displays can play it without making queue operation depend on it. Related: FR-MEDIA-002/003/004.

US-SET-001 – Configure the display

Status: IMPLEMENTED. As an operator, I want to adjust announcement time, page rotation, and footer messages so that the display matches daily operation. Related: FR-SET-001.

US-SEC-001 – Restrict operational changes

Status: NOT_IMPLEMENTED; stakeholder requirement not formalized. As an operator/owner, I may need mutations restricted to authorized staff. Related: FR-AUTH-001.

3.5 Acceptance criteria

AC-QUEUE – ready lifecycle

1. A value from 1 to 99 entered in the cashier UI is sent as a two-digit string and stored with a UUID, selected counter, status ready, equal creation/ready/last-announced timestamps, and a new rank.
2. The same request ID and payload returns the recorded response without another row or announcement; the same ID with different content returns conflict.
3. A distinct request for an already-active number returns conflict.
4. Recall raises the turn to the start of the ready list and produces one fresh announcement.
5. Delivery or cancellation removes the turn from snapshots; its number can then be used for a new UUID.

AC-SYNC – connected screens

1. Each Socket.IO connection receives one current state snapshot.
2. Each successful mutation broadcasts the resulting snapshot.
3. Only non-replayed create/announce mutations broadcast announcement.
4. Reconnection restores state without replaying an old announcement.

AC-MEDIA – local and optional connected media

1. Only supported files discovered below an allow-listed genre directory are served by /media.

2. A local selection is rejected if the playlist does not exist or has no tracks.
3. A YouTube selection is accepted only for the server's HTTPS host/identifier allow-list.
4. YouTube failure leaves the queue visible and shows local fallback content/notice.

AC-AUDIO — unresolved

The expected audible sequence is turn number then counter/ready suffix, with serialized announcements and media muted during the call. The code supplies those mechanics, but no public-screen control invokes `enableAudio()`; therefore the end-to-end acceptance criterion is currently not met.

3.6 Requirements traceability

Requirement	Story	Main implementation	API/event	Verification
FR-QUEUE-001	US-QUEUE-001	CashierPage; StateService.create	POST /api/turns	automated
FR-QUEUE-002	US-QUEUE-001	service + one_active_number	409 conflict	automated
FR-QUEUE-003	US-QUEUE-001	form, validation, update	POST/PATCH turns	automated backend
FR-QUEUE-004	US-QUEUE-002	StateService.announce	announce + state events	automated
FR-QUEUE-005	US-QUEUE-003/004	ReadyList; update	PATCH /api/turns/:id	automated backend
FR-QUEUE-008	US-QUEUE-001/002	requests table; replay/record	create/announce	automated
FR-DISPLAY-001	US-DISPLAY-001	PublicPage	state event	manual only
FR-DISPLAY-002	US-SET-001	PublicPage; topbar switch	settings + state	manual only
FR-AUDIO-001	US-AUDIO-001	announcements, audio, and media player	announcement event	incomplete
FR-MEDIA-001	US-MEDIA-001	MediaService; MediaPlayer	refresh; media GET	backend automated
FR-MEDIA-002	US-MEDIA-001/002	multimedia panel/service	multimedia PUT	automated backend

FR-MEDIA-003	US-MEDIA-002	URL parsers/validation	multimedia PUT	automated backend
FR-SYNC-001	all live stories	EventsGateway; useSystem	state and announcement	integration automated
FR-AUTH-001	US-SEC-001	none	all routes open	not implemented

“Automated backend” does not imply browser behavior is covered. Test identifiers and manual gaps are detailed in `07-quality/test-cases.tex`.

4 User experience

4.1 Interface overview

The frontend is one React bundle with pathname-based selection and in-component navigation. There is no routing library.

Path	Rendered interface	Evidence/notes
/pantalla	Public display	The only explicit public-display SPA route.
/	Cashier workspace	Electron opens this route.
/caja	Cashier workspace	Server alias; no separate component.
/admin	Cashier workspace	Alias only; no distinct admin interface or authorization.
/display	Cashier workspace	Despite its name, route logic does not render the public display.
Other client paths	Depends on hosting	App chooses cashier unless pathname begins with /pantalla; production server only declares the five SPA routes above.

The cashier workspace has three client-side sections: Turns, Multimedia, and Settings. Changing section updates React state but not the URL. The public display has no navigation and no queue mutation controls.

There is no login page, account management, history screen, search view, or dedicated administrator page. The historical README's claim that the cashier can search pending turns is not supported by the current UI.

Evidence: `src/App.tsx`; `src/pages/CashierPage.tsx`; `server/bootstrap.ts`

4.2 Cashier interface

Purpose and main user

The cashier workspace supports day-to-day ready-list operation and display configuration. Its main user is the queue operator; because authorization is absent, this is a UX role rather than a security identity.

Components and actions

- **Sidebar/top bar:** choose Turns, Multimedia, or Settings; open the public display; view local connection state; toggle automatic turn-page rotation.
- **New ready turn:** enter 1–2 digits, select optional counter 1 or 2, and submit. Successful creation clears/refocuses the number input.
- **Repeat panel:** select an active turn and explicitly reannounce it.
- **Ready list:** newest/recalled first, with an action dialog per turn. The dialog can save a changed counter, mark delivered, or enter a second confirmation step before cancellation.
- **Multimedia:** configure YouTube or local playlists, refresh the catalog, pause/resume, mute/unmute, change volume, or return to fallback visuals.
- **Settings:** set announcement duration and footer lines, test local announcement audio in the cashier browser context, and open the public display.

State and dependencies

The page reads snapshots and connection status from `SystemProvider`, then writes through `src/services/api.ts`. All mutation controls associated with `queue/media` state are intended to be disabled while disconnected. Feedback is shown in an inline live region on the Turns section and a toast elsewhere.

UX constraints and discrepancies

- The interface does not search or filter a long ready list.
- The announcement-duration input advertises maximum 20 seconds, while server validation permits 30.
- The audio test activates only the cashier tab's Web Audio context; it does not activate public-screen audio.
- Media/settings controls are effectively administrative but share the operator interface and authorization level.
- The responsive layout changes from full sidebar to rail to horizontal navigation at defined CSS breakpoints.

Evidence: `src/pages/CashierPage.tsx`; `src/components/ReadyList.tsx`; `src/components/SettingsPanel.tsx`; `src/components/MultimediaPanel.tsx`

4.3 Client/public display

Purpose and main user

The display lets waiting customers match a printed ticket with a ready number and, where assigned, a pickup counter. It is read-only at the component level.

Rendered regions

- a five-row ready-turn panel ordered newest/recalled first;
- “Último llamado” emphasis for the first turn and “Llamando ahora” for the active announcement;
- automatic eight-second pagination, or a manual next-page button when rotation is disabled;
- a media frame containing campus fallback images, a local-audio status overlay, or YouTube;
- a large assertive live-region announcement overlay for the configured duration;
- a continuous footer ticker with logos, date, time, and configured messages;
- connection, audio, and provider-failure notices when those states occur.

Behavior

A new or recalled turn resets pagination to page 1. Announcements are queued, deduplicated by announcement ID in memory, skipped if their turn has already closed, and cancelled on disconnect. Media is intended to mute during an announcement and resume afterward.

Current gaps

- **Audio activation:** `useAnnouncements` returns `enableAudio` and `testAudio`, but `PublicPage` does not use either. Its `audioEnabled` value remains false, so announcement playback is bypassed and local/YouTube media is kept muted.
- **Empty state:** no explicit “no ready orders” message is rendered; the list is simply empty.
- **Mascot:** `TrojanMascot.tsx` and its CSS/assets exist, but the component is not mounted.

- **Fallback naming:** the active implementation uses a campus image carousel, not the unused `coffee-welcome.png` described by older redesign documentation.
- **Verification:** no automated browser, visual-regression, accessibility, or real audio/playback test exists.

Evidence: `src/pages/PublicPage.tsx`; `src/hooks/useAnnouncements.ts`; `src/components/MediaPlayer.tsx`; `src/styles/public.css`

4.4 Administration interface

NOT_IMPLEMENTED as a distinct interface or role.

The production server serves `/admin`, but `App.tsx` renders the same `CashierPage` used at `/`. The “Multimedia” and “Configuración” sections provide administrative functions inside that page:

- select YouTube/local/fallback media and playback options;
- refresh the local music catalog;
- set announcement duration, page rotation, and footer messages;
- run a cashier-context audio test.

There are no administrator accounts, protected routes, privilege checks, user management, audit log, environment editor, backup controls, or deployment controls. Product owners must decide whether a distinct administrator is required. Until then, documentation uses “operator configuration” rather than claiming an administrator permission boundary.

Evidence: `src/App.tsx`; `src/pages/CashierPage.tsx`; `server/bootstrap.ts`; `server/app.controller.ts`

4.5 UX guidelines

Current conventions

- Spanish (Mexico) interface copy, two-digit tabular ticket numbers, and canonical “turno”/“mostrador” labels.
- Montserrat font files are bundled through `@fontsource`; icons are code-native SVG paths.
- Controls generally provide labels or accessible names, focus-visible outlines, disabled states, status/live regions, and a skip link.

- Dialogs use the native `dialog` element and require a second step for cancellation.
- Responsive CSS adapts cashier and public layouts; reduced-motion media queries suppress transitions/animations.
- Connection loss remains visible and mutation controls are disabled while cached state stays on screen.

Guidelines for changes

Preserve two independent identifiers (visible number and UUID), do not introduce preparation language, keep destructive cancellation confirmed, and ensure every successful-looking mutation is backed by a successful server response. New configuration must state whether it is local, persistent, per-browser, or broadcast.

Required validation

Viewing distance, minimum contrast/font size, screen orientation, browser/TV compatibility, keyboard-only behavior, assistive-technology behavior, touch targets, Spanish terminology, and reduced-motion acceptance all require formal testing. Existing CSS is evidence of intent, not proof of standards conformance.

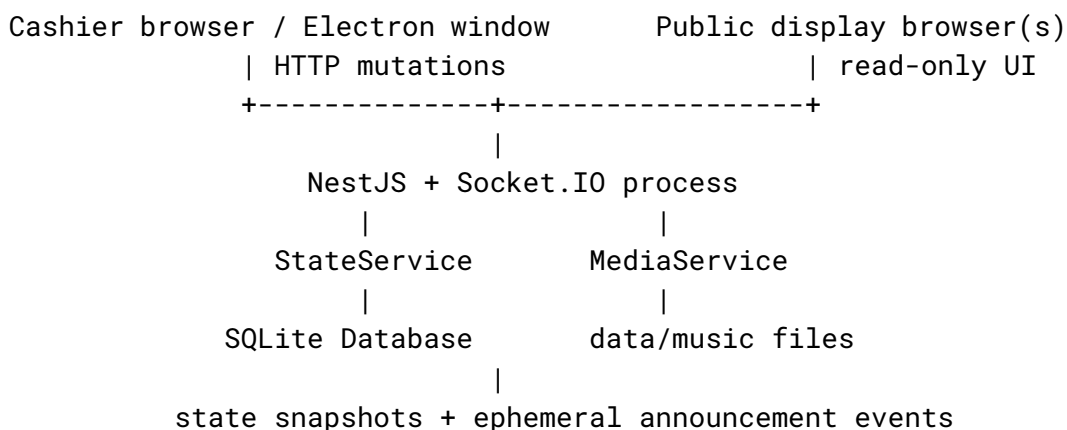
Historical redesign notes migrated

The removed `docs/UI-REDESIGN.md` described the Troyanos blue/white visual identity, glass surfaces, responsive layouts, public-display pagination, and local assets. Those claims were retained here only where the current source still supports them. Its statements that public audio and a coffee welcome image were operational are superseded by current code evidence.

5 Architecture

5.1 System overview

COMAL Queue is a TypeScript monorepository without workspace packages. One React bundle provides cashier and public views. One NestJS process serves JSON endpoints, Socket.IO, local media, and the production SPA. SQLite and filesystem audio are local process dependencies.



Optional: Public browser -> YouTube IFrame API (Internet)

Runtime modes

Development: `npm run dev` starts the Nest server on 127.0.0.1:3001 and Vite on 127.0.0.1:5173; Vite proxies API, media, and Socket.IO paths.

Built server: `npm run build` generates the SPA and server JavaScript; `npm start` serves both from port 3001 by default.

Electron: a single-instance Windows desktop shell starts the compiled server on loopback, uses an application user-data SQLite path, loads the cashier root route, and stops the child server on quit.

No cloud service, reverse proxy, container, authentication service, ORM, migration framework, job queue, analytics system, or observability platform is present.

Evidence: `package.json`; `vite.config.ts`; `server/bootstrap.ts`; `electron/main.cjs`

5.2 Frontend architecture

Framework and bootstrap

React 19 is mounted by `src/main.tsx` under Strict Mode. Vite handles development and bundling; Tailwind's Vite plugin is configured, but the UI is primarily authored in large custom CSS files rather than utility classes.

Routing and page structure

`App.tsx` selects `PublicPage` only when the pathname begins with `/pantalla`; all other paths render `CashierPage`. Cashier sections are a local page state, not routes.

Shared state and transport

`SystemProvider` owns the last server snapshot and `Socket.IO` connectivity. It connects with `WebSocket`-only transport, exposes a callback set for announcement events, and does not issue `GET /api/state`. Components call a small fetch-based API client for mutations. State changes arrive back through `Socket.IO` rather than optimistic updates.

Main responsibilities

Area	Responsibility
<code>pages/CashierPage</code>	navigation, create/recall forms, global feedback, connection handling
<code>pages/PublicPage</code>	queue paging, announcement overlay, media and footer composition
<code>components/ReadyList</code>	active list and counter/deliver/cancel dialog
<code>components/MultimediaPanel</code>	media selection and persisted playback controls
<code>components/SettingsPanel</code>	display settings and cashier-context audio test
<code>components/MediaPlayer</code>	YouTube lifecycle, local track playback, fallback, media ducking
<code>hooks/useAnnouncements</code>	ephemeral queue, duration, cancellation, audio sequence
<code>hooks/useSystem</code>	<code>Socket.IO</code> state and subscriber registry

<code>services/api</code>	HTTP serialization, eight-second timeout, message extraction
<code>styles</code>	shared tokens plus cashier/public responsive layouts

Error and loading states

Initial state is null until the first socket snapshot. Both pages render connection text; mutations are disabled when disconnected. API errors become Spanish messages. Media maintains separate offline/provider/blocked states. There is no error boundary, retry queue for mutations, or persistent operation log.

Type discrepancy

Frontend Turn and Announcement declare timestamps as numbers, while the server emits ISO strings. Runtime rendering currently avoids arithmetic on these fields, but the contract is incorrect and not caught because client/server types are duplicated.

Evidence: `src/; vite.config.ts; tsconfig.json`

5.3 Backend architecture

Bootstrap and module

`server/main.ts` validates PORT, selects HOST, and starts a NestJS 11 Express application. `bootstrap.ts` enables CORS, shutdown hooks, production static assets, and five explicit SPA routes. `AppModule` wires two controllers and four singleton providers.

Layers

- **Controllers:** map HTTP routes and broadcast after successful service mutations. No authentication decorators or API version prefix exist.
- **Validation functions:** accept unknown bodies, reject arrays/unknown fields, and validate every supported primitive/domain input without `class-validator`.
- **StateService:** owns queue lifecycle, ordering, idempotency, multimedia/settings invariants, and snapshot/history queries.

- **LocalDatabase:** opens `node:sqlite DatabaseSync`, creates the current schema at startup, exposes prepared statements and a transaction helper.
- **MediaService:** discovers genre directories/files, builds a cloned catalog, and resolves only previously discovered files.
- **EventsGateway:** emits a snapshot at connect and broadcasts state plus optional announcement events.

Dependency direction

Controllers depend on `StateService`, `EventsGateway`, and `MediaService`. `StateService` depends on `LocalDatabase` and `MediaService`. `EventsGateway` depends on `StateService`. `MediaService` and `LocalDatabase` depend only on Node APIs. There is no repository abstraction, ORM, CQRS/event store, or domain-module separation.

Concurrency model

Database writes are synchronous and wrapped in `BEGIN IMMEDIATE`; this serializes writers inside the Node process/SQLite file. There is no distributed lock or multi-process coordination beyond SQLite. Reusing one database from multiple packaged processes is not documented or tested.

Evidence: `server/; tsconfig.server.json`

5.4 Data flow

Create/recall mutation

```

UI -> fetch JSON -> Controller -> StateService
validation/invariants
  -> BEGIN IMMEDIATE -> turns + requests -> COMMIT
  -> Controller -> Socket.IO state -> every client
                        + announcement -> every live client
  <- HTTP turn response

```

The HTTP response confirms the mutation to the caller; client lists update from the separately broadcast state. An exact retry returns the stored response and deliberately omits a second broadcast announcement.

Other mutation

Counter/status, multimedia, settings, and catalog refresh operations mutate SQLite or the in-memory catalog, then broadcast a full snapshot. They do not emit announcement events.

Connection/reconnection

When a socket connects, the gateway immediately calculates and emits a full snapshot. The frontend replaces its current state. No prior announcement is replayed, because announcements are not selected from SQLite during connection.

Local media request

Catalog refresh maps genre/filename keys to canonical real paths. The browser requests an encoded /media/:genre/:filename URL. The controller asks MediaService for the mapped path and Express sends that file. Uncatalogued, unsupported, removed, or traversal-like paths return not found.

YouTube flow

The cashier validates a URL client-side, then the server applies a stricter HTTPS allow-list and persists it. Each public browser loads the remote IFrame API directly and manages its own player. Playback position is not synchronized between screens or persisted.

5.5 Database

Technology and lifecycle

The backend uses Node's experimental built-in `node:sqlite DatabaseSync` API. Default path is `data/comal.sqlite`; Electron overrides it with `app.getPath('userData')/comal.sqlite`. Parent directories and tables/indexes are created during provider construction. There are no versioned migrations or seed records.

Logical model

turns	requests	configuration
-----	-----	-----
id (PK)	id (PK)	key (PK)
number	kind	value (JSON text)
counter	fingerprint	
status	response (JSON text)	
timestamps	created_at	
rank		

No declared foreign-key relationships.

requests.response embeds a turn and announcement; requests has no FK to turns.

Turns table

Column	Type/constraint	Meaning
id	TEXT primary key	Server-generated UUID.
number	TEXT, two digits, integer 1–99	Visible ticket number, including leading zero.
counter	INTEGER in 0,1,2; default 0	No assigned counter, counter 1, or counter 2.
status	TEXT in ready, delivered, or cancelled	Ready is active; other values are terminal in service logic.
created_at	TEXT not null	ISO time at creation.
ready_at	TEXT not null	Same creation time; never subsequently changed.
last_announced_at	TEXT not null	Creation or latest recall time.
rank	INTEGER not null	Display/history ordering; incremented at create and recall.

Indexes are one_active_number, a unique partial index on number where status is ready, and turn_display_order on status plus descending rank.

Configuration and requests

configuration stores JSON under multimedia and settings; defaults are code constants and are not written until changed. requests stores

idempotency kind, input fingerprint, serialized mutation result, and timestamp. Neither table has retention or corruption recovery logic.

Durability and recovery characteristics

WAL mode, foreign keys, and a 5000 ms busy timeout are enabled. Writes in StateService use a transaction helper. No schema version, backup, restore, integrity-check command, compaction policy, encryption, or automated recovery exists.

Evidence: `server/database.ts`; `server/state.service.ts`; `electron/main.cjs`

5.6 Turn and queue domain

State model

```
create
|
v
[ready] ---- mark delivered ----> [delivered]
|
+----- cancel capture -----> [cancelled]
```

Recall: `ready -> ready` (new rank, timestamp, and announcement)

Counter edit: `ready -> ready`

Closed states have no implemented outgoing transition.

Actual states: ready, delivered, and cancelled. There is no created, waiting, preparing, announced, completed, removed, or expired state. “Removed” in UI means cancelled; records are not deleted.

Generation and identity

The cashier supplies the visible number; the UI trims, range-checks, and pads it. The server accepts only the exact two-character forms 01–99. It generates a UUID for the turn and a separate UUID for each announcement. Request IDs are normally browser-generated UUIDs.

Ordering

`nextRank()` selects maximum historical rank plus one. Creation and recall receive that rank. Active snapshots sort ready turns by descending rank, so recall moves a turn to the front. History uses the same rank ordering and a 200-row limit, not closure or creation time.

Persistence and duplicate prevention

Ready turns remain through page reload and process restart. Duplicate active numbers are rejected by a pre-insert query and a partial unique index. After delivery/cancellation, the same visible number can create a distinct UUID. Exact create/recall retries reuse a recorded response; mismatched reuse of a request ID returns conflict.

Client awareness

Every mutation emits a full snapshot. New/recall actions also emit a transient announcement to clients connected at that moment. Reconnecting clients learn current ready state but receive no missed audio/visual announcements.

Closure metadata

Delivery and cancellation only change status. There is no delivered/cancelled timestamp, actor, reason, audit event, or soft-delete marker beyond status. This limits operational auditability.

Evidence: `server/models/state.ts`; `server/state.service.ts`; `server/database.ts`; `tests/backend.test.ts`

5.7 Realtime communication

The application uses Socket.IO with WebSocket-only frontend transport. It does not use polling, SSE, Redis, or a message broker.

Event	Producer	Consumer	Payload/purpose
-------	----------	----------	-----------------

state	EventsGateway	all frontend views	Full AppState: active turns, multimedia, catalog, settings. Emitted on connection and after every successful mutation/refresh.
announcement	EventsGateway	live frontend views	Announcement ID, turn ID, number, counter, ISO creation time. Emitted only after non-replayed create/recall.
connect	Socket.IO client runtime	SystemProvider	Marks the client connected; state normally follows from gateway connection handling.
disconnect / connect_error	Socket.IO client runtime	SystemProvider	Marks disconnected; announcement queue is cleared by the public hook.

Connection lifecycle

The client enables reconnection with delay starting at 800 ms and capped at 5 seconds. No custom connection authentication, namespace, room, heartbeat setting, maximum reconnect count, or application acknowledgement is defined. Socket.IO defaults apply where the repository is silent.

Ordering and failure behavior

Full-state broadcasts reduce incremental reconciliation complexity. Announcement delivery is best effort: it is not persisted as an event stream, acknowledged, or replayed. The public hook serializes received announcements, discards duplicate IDs in a bounded in-memory set, skips events whose turn has closed, and aborts on disconnect. Multiple server processes would not share gateway broadcasts.

Test evidence

The integration test connects two clients, verifies initial snapshots, one announcement for an idempotent retry, recall ordering, media state broadcast, reconnect restoration without old audio, and close propagation.

Evidence: `server/events.gateway.ts`; `src/hooks/useSystem.tsx`; `src/hooks/useAnnouncements.ts`; `tests/backend.test.ts`

5.8 Multimedia architecture

Modes

Fallback Persisted type with null URL/playlist and forced `playing=false`. The current public implementation shows a looping campus image carousel.

Local Persisted playlist ID references the server's current filesystem catalog. The browser streams one file at a time, advances on ended, and restarts at track zero after media selection changes.

YouTube Persisted URL identifies a video or playlist. The public client loads the remote IFrame API, creates its own player, loops it, and exposes `retry/error/autoplay-blocked` behavior.

Local catalog

Built-in genres are `lo-fi`, `jazz`, and `rock`; they appear even when empty. Additional directory names matching `[a-z0-9_-]` (1–64 characters) appear only when they contain a supported extension: MP3, WAV, OGG, M4A, AAC, or FLAC. Files sort by Spanish locale. Canonical real-path checks and a catalog map prevent arbitrary filesystem reads.

Six short generated demo WAVs are committed across the three built-in genres. The repository also commits 99 number announcements plus three suffixes. Generation is optional developer tooling; runtime does not download speech assets.

Playback, fallback, and synchronization

Multimedia selection/playing/volume/mute is shared via SQLite and snapshots. Track index, YouTube position, provider readiness, online status, and autoplay state are per browser and not synchronized. The fallback carousel remains visible for local-audio mode and whenever YouTube is inactive or unavailable. YouTube requires Internet; queue UI and local state remain usable on provider failure.

Announcements and ducking

The intended call sequence is `/audio/turns/NN.wav` followed by either `/audio/counters/1|2.wav` or `/audio/ready.wav`. `MediaPlayer`

mutes during a current call. This end-to-end feature is `PARTIALLY_IMPLEMENTED` because `PublicPage` never activates its Web Audio context or changes `audioEnabled` from false.

Validation discrepancy

The frontend parser accepts HTTP or HTTPS and returns a playlist in preference to a co-present video. The server rejects non-HTTPS and has slightly different identifier bounds. Server validation is authoritative, but the mismatch can produce client/server error differences.

Evidence: `server/media/media.service.ts`; `server/dto/validation.ts`; `src/components/MediaPlayer.tsx`; `tooling/audio/`; `data/music/`

5.9 Authentication and authorization

`NOT_IMPLEMENTED`.

There is no login page, user table, password, hash library, session, token, cookie, identity provider, Nest guard, role decorator, route middleware, or protected frontend route. The public display is read-only only by presentation; it shares network access to unauthenticated mutation endpoints.

Capability	Operator UI	Public UI	API enforcement
View active state	Yes	Yes	None
View up to 200 historical turns	No	No	Public endpoint
Create/recall/update turns	Yes	No	None
Configure media/settings	Yes	No	None
Refresh media catalog	Yes	No	None
Read catalogued media	Indirect	Yes	None

An authorization policy cannot be invented from current code. Stakeholders must determine whether trusted loopback/physical access is sufficient or whether a LAN deployment requires operator authentication and/or network segmentation. Until then, no document should describe Cashier or Administrator as enforced roles.

Evidence: `package.json`; `server/app.controller.ts`; `server/app.module.ts`; `src/App.tsx`

5.10 Observed architectural decisions

These are retrospective records of implemented choices. Original rationale is not documented unless stated.

AD-001 – Local SQLite persistence

Decision: use synchronous `node:sqlite` with a local file. **Observed context:** local-first queue and Electron target. **Original rationale:** not documented. **Implications:** simple deployment and restart persistence; experimental Node API, single-file recovery concerns, synchronous access, no migrations.

AD-002 – Snapshot-based realtime state

Decision: broadcast complete state after mutations and on connect, plus separate ephemeral announcements. **Observed context:** small state shape and multiple displays. **Original rationale:** not documented. **Implications:** easy reconnection; payload grows with active turns/catalog and announcements are best-effort.

AD-003 – Ready-only domain

Decision: create turns directly in ready; closed states are delivered/cancelled. **Observed context:** ticket creation and preparation occur outside the system. **Original rationale:** explicitly stated in prior README. **Implications:** small workflow; no preparation metrics or upstream integration.

AD-004 – Idempotency by caller request ID

Decision: persist request kind, fingerprint, and response for create/recall. **Observed context:** prevent double-click/network retry duplicates and duplicate calls. **Original rationale:** prior requirements and code comments imply retry safety. **Implications:** deterministic retry behavior; unbounded request table and no expiry.

AD-005 – Client-side multimedia players

Decision: persist configuration centrally but run local/YouTube playback independently in each public browser. **Original rationale:** not documented.

Implications: simple server and optional provider; screens are not position-synchronized and autoplay/audio permission is per browser.

AD-006 – One SPA for operator and public views

Decision: select the page by raw pathname, with no router or route guards.

Original rationale: not documented. **Implications:** minimal dependency; route aliases are misleading and administration is not separated.

AD-007 – Windows desktop packaging

Decision: Electron starts the built server on loopback and loads the cashier root in a single window. **Original rationale:** not documented. **Implications:** local single-instance operation; remote displays require a different topology or mirrored output.

6 HTTP API

6.1 API overview

The NestJS application exposes unversioned JSON routes below `/api` and file delivery below `/media`. There is no OpenAPI/Swagger definition, authentication, rate limiting, pagination metadata, request correlation header, or documented stability policy.

Conventions

- JSON request bodies use `Content-Type: application/json`; validators reject non-objects, arrays, and unknown fields.
- The frontend uses relative URLs, an eight-second timeout, and displays the server's string message when available.
- POST creation returns Nest's default 201; recall and playlist refresh explicitly return 200. GET, PUT, and PATCH use 200 on success.
- Turn IDs in paths are UUID strings in normal operation, but path syntax itself is not validated before database lookup.
- All routes are unauthenticated and have no enforced role.

Base URLs

Development frontend requests are proxied from `http://127.0.0.1:5173` to `http://127.0.0.1:3001`. A built server serves API and UI from port 3001 by default. Host and port are configurable outside the Electron launcher.

Realtime complement

HTTP mutations return their direct result, while every successful mutation also publishes a fresh state Socket.IO event. Create and recall publish an additional announcement unless the request is an idempotent replay.

Evidence: `server/app.controller.ts`; `server/bootstrap.ts`; `src/services/api.ts`; `vite.config.ts`

6.2 Endpoints

GET /api/health

Authentication/authorization: none. **Purpose:** process readiness probe. **Response:** "ok": true, "service": "comal-local". **Related code:** `AppController.health`; used by Electron startup.

GET /api/state

Authentication/authorization: none. **Purpose:** return current ready turns, multimedia configuration, discovered playlists, and settings. **Response:** `AppState`. **Notes:** the current frontend obtains this data from `Socket.IO` rather than this route.

GET /api/history

Authentication/authorization: none. **Purpose:** return up to 200 turns of all statuses, ordered by rank descending. **Parameters:** none; no pagination/filtering. **Response:** `Turn[]`. **Related code:** `StateService.history`.

POST /api/turns

Authentication/authorization: none. **Purpose:** create a turn directly in ready state and announce it. **Body:** number (required exact two-digit string 01–99), counter (optional 0/1/2, default 0), `requestId` (required 8–100-character allow-listed string). **Success:** 201 with `Turn`. **Errors:** 400 invalid/unknown fields; 409 active duplicate or mismatched request-ID reuse. **Side effects:** transactionally writes turn/request; emits state and, unless replayed, announcement.

POST /api/turns/:id/announce

Authentication/authorization: none. **Purpose:** recall an active ready turn. **Body:** required `requestId`. **Success:** 200 with updated `Turn`. **Errors:** 400 invalid body; 404 unknown turn; 409 closed turn or mismatched request-ID reuse. **Side effects:** new rank/timestamp and request record; emits state and, unless replayed, announcement.

PATCH /api/turns/:id

Authentication/authorization: none. **Purpose:** change counter and/or status. **Body:** at least one of counter (0/1/2) and status (ready/delivered/cancelled); unknown fields rejected. **Success:** 200 with Turn. **Errors:** 400 invalid/empty body; 404 unknown turn; 409 attempt to change a closed turn. **Side effects:** updates turn and emits state; no announcement.

PUT /api/multimedia

Authentication/authorization: none. **Purpose:** merge and persist multimedia configuration. **Body:** non-empty subset of type, url, playlistId, playing, volume, muted. **Success:** 200 with normalized Multimedia. **Errors:** 400 for invalid type/value, invalid YouTube URL, missing/empty local playlist. **Normalization:** YouTube clears playlist ID; local clears URL; fallback clears both and forces stopped. **Side effects:** persists JSON and emits state.

PUT /api/settings

Authentication/authorization: none. **Purpose:** merge and persist public-display settings. **Body:** non-empty subset of integer announcementSeconds (3–30), boolean autoRotate, and footerMessages (1–10 nonblank strings, each at most 120 characters). **Success:** 200 with Settings; messages are trimmed. **Errors:** 400 invalid/unknown/empty input. **Side effects:** persists JSON and emits state.

POST /api/playlists/refresh

Authentication/authorization: none. **Purpose:** rescan configured music directories. **Body:** ignored; frontend sends an empty object. **Success:** 200 with Playlist[]. **Side effects:** replaces in-memory catalog and emits state; does not persist catalog rows.

GET /media/:genre/:filename

Authentication/authorization: none. **Purpose:** stream a file already present in the MediaService allow-list. **Success:** Express sendFile response. **Errors:** 404 if key is absent or file no longer exists. **Security boundary:** route values

are used only as a lookup key; the response path comes from the canonical catalog map.

Evidence: `server/app.controller.ts;` `server/state.service.ts;`
`server/dto/validation.ts;` `server/media/media.service.ts`

6.3 Data contracts

All timestamps emitted by the server are ISO 8601 strings created by `new Date().toISOString()`.

Turn

```
{
  id: string,                // UUID
  number: string,            // "01" through "99"
  counter: 0 | 1 | 2,
  status: "ready" | "delivered" | "cancelled",
  createdAt: string,         // ISO date-time
  readyAt: string,           // ISO date-time
  lastAnnouncedAt: string,  // ISO date-time
  rank: number               // positive monotonic integer
}
```

Listing 1: Turn response contract

Announcement

```
{
  id: string, turnId: string, number: string,
  counter: 0 | 1 | 2, createdAt: string
}
```

Listing 2: Announcement event contract

Multimedia and settings

Server defaults are fallback media, stopped, volume 65, unmuted; announcement duration 8 seconds; automatic rotation enabled; and three Spanish footer messages.

```

Multimedia = {
  type: "fallback" | "youtube" | "local",
  url: string | null,
  playlistId: string | null,
  playing: boolean, volume: number, muted: boolean
}

Settings = {
  announcementSeconds: number,
  autoRotate: boolean,
  footerMessages: string[]
}

```

Listing 3: Multimedia and settings contracts

Playlist catalog

```

Track = { id: string, title: string, url: string }
Playlist = { id: string, name: string, cover: string, tracks:
Track[] }
AppState = { turns: Turn[], multimedia: Multimedia,
  playlists: Playlist[], settings: Settings }

```

Listing 4: Playlist catalog and application-state contracts

Contract debt

`src/types.ts` incorrectly declares `turn/announcement` timestamps as numbers and makes `Settings.autoRotate` optional, while server contracts use ISO strings and require the field in snapshots. Client and server models are duplicated. No schema generation or runtime response validation exists.

Evidence: `server/models/state.ts`; `server/state.service.ts`; `src/types.ts`

6.4 Error handling

Backend

Validation and domain failures use Nest exceptions:

- `BadRequestException` (400): malformed bodies, unknown fields, invalid ranges/identifiers/provider URLs, unavailable local playlist;
- `NotFoundException` (404): unknown turn or unlisted/missing local file;
- `ConflictException` (409): duplicate active number, incompatible request-ID reuse, or mutation/recall of a closed turn.

Nest's default exception serialization is relied upon; no repository-defined error code, envelope, localization contract, or global exception filter exists. Unexpected SQLite/filesystem/programming errors become framework 500 responses and server logs. Transactions roll back synchronous exceptions.

Frontend HTTP

Requests abort after eight seconds. The client attempts JSON parsing and displays a returned string message; otherwise it uses a generic local-connection message. There is no retry except user repetition, no offline mutation queue, and no differentiation among validation, conflict, timeout, and server failure beyond text.

Realtime

Disconnect/connect-error sets a visible offline state. The last snapshot stays in memory, cashier mutations are disabled, the public display warns about staleness, and pending announcement audio is aborted. Reconnection replaces state. There is no durable missed-event recovery, but state itself is restored.

Multimedia

YouTube load timeout, script failure, provider errors, embed restriction, autoplay block, and offline events produce fallback UI; some allow retry. Local audio element failure produces a notice. Queue state remains independent. Public audio activation failure is not currently surfaced because activation is never attempted there.

Inconsistencies

Client and server YouTube validation differ; settings duration UI caps 20 while the server caps 30; no global React error boundary exists; `MediaService`

startup filesystem failures can prevent server start. These are registered as technical debt.

Evidence: `server/dto/validation.ts;` `server/state.service.ts;`
`src/services/api.ts;` `src/hooks/useSystem.tsx;` `src/components/`
`MediaPlayer.tsx`

7 Quality

7.1 Quality strategy

Current controls

- strict TypeScript configurations for frontend and backend;
- build command combining frontend type check, Vite production build, and backend compilation;
- five Node test-runner tests covering domain, persistence, validation, local catalog, real HTTP, and Socket.IO;
- database constraints plus service validation and transactions;
- an audio verifier for count, RIFF/WAVE integrity, format, duration, and non-silence;
- idempotency and duplicate-active-number controls;
- explicit offline/provider fallback UI and reduced-motion CSS.

On 14 September 2026 at commit bf71e73, all five backend tests passed, both TypeScript projects passed `--noEmit` checks, the production build succeeded, and the audio verifier confirmed 102 announcement assets (99 numbers plus three suffixes) and six demo music tracks. The first sandboxed test launch failed before discovery due to a host user-profile memory error; rerunning outside that restricted environment passed. This was an execution-environment issue, not a failing test.

Missing quality gates

There is no CI, lint script/configuration, formatter configuration, coverage tool/threshold, frontend unit/component test, automated end-to-end browser test, accessibility scan, visual regression, load test, security scan, dependency update policy, or release acceptance record.

Recommended evidence order

P0: add a browser test for the complete ready/visual/audio/close flow and fix audio activation. P1: run current tests/type/build/audio verification in CI, then cover cashier/public components and recovery. P2: add accessibility, visual viewport, compatibility, and load tests after stakeholders define targets.

7.2 Testing strategy

Current automated testing

`npm test` executes `tests/backend.test.ts` through `tsx` and Node's test runner. The five tests cover:

1. ready lifecycle, ordering, duplicate/retry behavior, counter edit, closure, and number reuse;
2. malformed payloads and media/settings validation without unintended mutation;
3. SQLite persistence of turns, ordering, idempotency, media, and settings after reopen;
4. local catalog discovery and rejection of unlisted/traversal-like paths;
5. a real Nest HTTP server with two WebSocket clients, state/announcement broadcast, recall ordering, media broadcast, reconnect, close, and health.

`node tooling/audio/verify-audio.mjs` is a standalone asset check, not part of `npm test`. TypeScript compilation is enforced by `npm run build` but no type-check-only `npm` script exists.

Current manual testing evidence

The superseded redesign note claimed manual viewport, offline, media, YouTube fallback, queue action, and audio activation checks. Those results are historical and not reproducible from a committed test script/report. Actual YouTube playback was explicitly unverified. They are not treated as current automated assurance.

Critical untested workflows

Public audio activation/playback, React rendering/actions, public empty state, pagination, media duck/resume, autoplay policy, multiple real screens, browser compatibility, Electron install/upgrade, LAN exposure, corrupted/locked database recovery, large queues/history, settings UI/server-boundary mismatch, accessibility, and backup/restore.

Recommended layers

Retain service/integration tests; add component tests for validation and controls, browser tests for two synchronized pages and audio activation with mocked media, packaged-app smoke tests, and operational recovery tests. Add load/security tests only after requirements are defined.

How tests are run

```
npm ci
npm test
npx tsc --noEmit -p tsconfig.json
npx tsc --noEmit -p tsconfig.server.json
node tooling/audio/verify-audio.mjs
npm run build
```

The final build command writes generated output; the preceding TypeScript commands are non-emitting checks.

7.3 Critical test cases

TC-QUEUE-001 – Create and broadcast a ready turn

Preconditions: empty test database; two connected clients. **Steps:** POST valid number/counter/request ID; observe response and both sockets. **Expected:** 201, one ready row, state on both clients, one announcement. **Automation:** automated.

TC-QUEUE-002 – Reject active duplicate and allow closed reuse

Preconditions: number 01 ready. **Steps:** create 01 with another request ID; deliver original; create 01 again. **Expected:** first attempt 409; later creation succeeds with new UUID. **Automation:** automated at service level.

TC-QUEUE-003 – Idempotent request handling

Steps: submit identical create twice with one request ID, then reuse that ID with a different payload. **Expected:** exact replay returns same turn/announcement without second broadcast; mismatch returns 409. **Automation:** automated (broadcast checked for create).

TC-QUEUE-004 – Recall ordering

Preconditions: two ready turns. **Steps:** recall older turn. **Expected:** it receives newest rank/timestamp, sorts first, and emits one fresh announcement. **Automation:** automated.

TC-QUEUE-005 – Counter update and closure

Steps: change a ready turn to counter 1, mark delivered, then attempt a different update. **Expected:** counter is broadcast; delivered leaves snapshot; later mutation conflicts. **Automation:** service lifecycle automated; full UI manual.

TC-SYNC-001 – Reconnect without repeated call

Steps: create turns, disconnect a client, reconnect. **Expected:** restored snapshot contains ready turns and no historical announcement is emitted. **Automation:** automated.

TC-PERSIST-001 – Restart persistence

Steps: write turns/media/settings, close SQLite, reopen. **Expected:** snapshot and idempotency response remain equal. **Automation:** automated.

TC-MEDIA-001 – Catalog allow-list

Steps: add supported/unsupported files and custom genre, refresh, request listed and traversal-like keys. **Expected:** supported canonical files appear; unsupported/unlisted paths do not resolve. **Automation:** automated with fixture files.

TC-MEDIA-002 – YouTube fallback

Steps: configure valid YouTube, simulate offline/script/provider/blocked states. **Expected:** queue remains visible; carousel/notice appears; retry is offered where applicable. **Automation:** recommended browser test; current status manual/historical only.

TC-AUDIO-001 — Public announcement sequence

Steps: activate audio on public display, create two rapid turns with/without counters. **Expected:** calls play once, serially, number then suffix; media ducks and resumes. **Automation:** blocked by current missing public activation; required after fix.

TC-AUTH-001 — Prevent unauthorized mutation

Expected requirement: requires stakeholder definition. **Current result:** every caller with network access can mutate. **Automation:** not applicable until authentication/authorization is implemented.

TC-RECOVERY-001 — Backup and restore

Steps/expected: TBD pending an approved WAL-aware backup procedure and recovery objectives. **Automation:** not implemented.

7.4 Known limitations

1. **Public audio is unreachable.** The hook exposes activation, but the public page neither calls it nor renders a control. All public media remains muted through the same flag.
2. **No authentication or authorization.** Network-level trust is the only effective boundary.
3. **No explicit public empty state.** An empty queue renders no explanatory message, contrary to prior requirements text.
4. **No history UI.** The API exposes 200 rows, without pagination/filtering/export or closure metadata.
5. **Loopback Electron server.** The packaged app does not directly expose the server to separate LAN devices.
6. **Playback is per browser.** Track position and YouTube state are not synchronized; autoplay rules vary by browser.
7. **Best-effort announcements.** Disconnected screens do not receive missed calls after reconnect.
8. **Unbounded persistence.** Turns and idempotency requests are never pruned; history merely limits query results.
9. **Schema is startup DDL.** No migration/version strategy exists for upgrades.

10. **Synchronous experimental SQLite API.** Compatibility depends on the pinned Node generation and write work blocks the event loop.
11. **Frontend contracts drift.** Timestamp types conflict with the server.
12. **Settings boundary mismatch.** UI maximum announcement duration is 20 seconds; server maximum is 30.
13. **No frontend/package automation.** Visual, audio, accessibility, and Electron behavior rely on manual validation.
14. **No operational tooling.** Backup, restore, monitoring, log rotation, graceful service supervision, and incident procedures are absent.
15. **Brand/asset drift.** COMAL, Comal++, and Troyanos coexist; mascot/reference/coffee assets or metadata are unused by the current render tree.

7.5 Technical debt register

ID	Area	Problem/impact	Priority	Suggested resolution
TD-001	Audio	Public activation path is not wired; audible calls and media sound do not work.	P0	Add explicit user activation on each public screen and browser tests for calls/ducking.
TD-002	Security	All state/history/media mutations are unauthenticated.	P0 if LAN-exposed	Define trust model, then implement authorization or constrain network/host.
TD-003	Recovery	No backup/restore procedure for SQLite/WAL.	P0 before live use	Define RPO/RTO; implement and exercise WAL-safe backup/restore.
TD-004	Tests	No frontend, E2E, accessibility, or packaging automation.	P1	Add browser/component tests and CI gates.
TD-005	Schema	Startup DDL has no schema version/migrations.	P1	Introduce migration tracking before schema evolution.
TD-006	Retention	turns/requests grow indefinitely; history hides records after 200.	P1	Define retention, archival, and idempotency-key expiry.

TD-007	Contracts	Duplicated client/server types disagree on timestamps and optional settings.	P1	Share or generate schema/contracts; validate responses.
TD-008	Audit	Closure records lack time, actor, and reason.	P1	Define audit requirement, then extend model/migration.
TD-009	Deployment	Electron loopback conflicts with direct multi-device LAN use.	P1	Select mirrored-display or secured LAN topology and package accordingly.
TD-010	Validation	Frontend/server YouTube rules and duration maxima differ.	P2	Derive both from shared validation/contract.
TD-011	Architecture	All domains/providers are in one Nest module and one state service.	P2	Separate only as change pressure warrants; preserve transaction invariants.
TD-012	Error handling	No stable error codes or frontend error boundary.	P2	Add an error contract and recoverable UI boundary.
TD-013	Operations	Only console logs/health exist; no metrics, alerting, rotation, or supervision.	P1	Define local operations baseline and add appropriate telemetry.
TD-014	UX	Public empty state is missing; long cashier list has no search/filter.	P2	Validate workflows and implement targeted states.
TD-015	Assets/brand	Unused components/assets and inconsistent names cause documentation/build noise.	P3	Decide canonical brand and remove/archive unused assets after owner approval.
TD-016	Tooling	No lint/format configuration and source formatting is inconsistent.	P2	Adopt and automate one TypeScript/CSS formatting standard.

Priorities express engineering risk, not business commitments. TD-002 depends on whether the service is confined to loopback or exposed on a shared network.

8 Development process

8.1 Development setup

Prerequisites

- Node.js 22.13.0 or newer (the audit used 22.19.0); node:sqlite is required.
- npm and the committed package-lock.json (lockfile version 3).
- A modern browser for the two web views. Internet is needed only to exercise YouTube or download dependencies/tools.
- Windows is required for the configured NSIS packaging target and PowerShell announcement generator; ordinary web development is not explicitly restricted by configuration.

Install and run both development processes

```
npm ci
npm run dev
```

This starts Nest at `http://127.0.0.1:3001` and Vite at `http://127.0.0.1:5173`. Open the latter root for the cashier and `/pantalla` for the public display. Vite proxies `API/media/Socket.IO` to Nest.

Separate commands are available:

```
npm run dev:server
npm run dev:client
```

Database and local media

No manual initialization is required. The server creates `data/comal.sqlite`, tables, indexes, and built-in music directories at startup. Override data locations with environment variables when isolation is needed. Committed demo music is already available.

To add local music, put supported files below `data/music/<genre>/` and invoke refresh from the UI. Supported extensions are MP3, WAV, OGG, M4A, AAC, and FLAC.

Checks

```
npm test
```

```
npx tsc --noEmit -p tsconfig.json
npx tsc --noEmit -p tsconfig.server.json
node tooling/audio/verify-audio.mjs
```

There is no lint or format command. All four checks were validated during this audit.

Build and run

```
npm run build
npm start
```

The build checks frontend types, creates `dist/`, and compiles the server into `build/`. The start command serves the combined app at loopback port 3001 by default. `npm run start:server` runs the TypeScript server directly and expects an existing `dist/` if the SPA is needed.

For desktop development after a build:

```
npm run build
npm run desktop
```

For Windows artifacts:

```
npm run desktop:dir
npm run desktop:build
```

These run electron-builder; signing, installer acceptance, and clean-machine verification are not configured/documented.

Regenerating optional audio assets

Runtime uses committed WAV files and does not require a synthesizer. On Windows, announcements can be regenerated with an existing eSpeak NG executable:

```
pwsh -ExecutionPolicy Bypass -File \
  tooling/audio/generate-announcements.ps1 \
  -EspeakPath 'C:\path\to\espeak-ng.exe'
```

The optional `-PrepareEngine` downloads pinned eSpeak NG 1.52.0, verifies its SHA-256, and extracts it locally using 7-Zip; it requires Internet but does not install the engine. Demo music is deterministic:

```
node tooling/audio/generate-demo-music.mjs
```

node tooling/audio/verify-audio.mjs

Regeneration changes committed binary assets and should be reviewed deliberately.

8.2 Project structure

Path	Responsibility
src/main.tsx	React bootstrap and global style/font imports.
src/App.tsx	Raw-path selection between cashier and public page.
src/pages/	Page composition and workflow orchestration.
src/components/	Queue, media, settings, footer, modal, branding, icon, and decorative UI.
src/hooks/	Socket state, announcement queue/audio, and localized clock.
src/services/	HTTP client, YouTube integration, local announcement playback.
src/styles/	Shared, cashier, and public CSS; responsive/reduced-motion behavior.
server/main.ts	Process configuration and listen.
server/bootstrap.ts	Nest creation, CORS, static assets, SPA route declarations.
server/app.controller.ts	API and media endpoint mapping.
server/state.service.ts	Queue domain, idempotency, state/config persistence.
server/database.ts	SQLite connection, schema DDL, transaction/config helpers.
server/events.gateway.ts	Socket.IO snapshots and announcement broadcast.
server/dto/validation.ts	Manual body/domain validation.
server/media/	Local playlist discovery and file allow-list.
server/models/	Backend data interfaces.
tests/backend.test.ts	All current automated tests.
public/assets/	Bundled images/logos/favicon; some reference/mascot assets are unused.
public/audio/	99 turn clips, two counter clips, and no-counter ready suffix.
data/music/	Demo and operator-provided local playlists plus manifest.

<code>electron/main.cjs</code>	Desktop process and embedded server lifecycle.
<code>scripts/dev.mjs</code>	Parallel dev server/client launcher.
<code>tooling/audio/</code>	Audio generation and structural verification.
<code>docs/</code>	This LaTeX product/process documentation system.
<code>build-tests/</code>	Committed compiled test/server output; not used by npm test.

Generated/ignored locations include `node_modules/`, `build/`, `dist/`, `release/`, SQLite files, logs, and test results, although historical tracked output may remain visible despite ignore rules.

8.3 Coding standards

Current conventions

- Strict TypeScript, ES2022 target, ESM for application code, and NodeNext with explicit `.js` relative imports in backend TypeScript.
- React function components and hooks; no class components or state library beyond context/local state.
- Nest decorators at controller/module boundaries; domain validation is implemented with pure functions receiving unknown.
- Prepared SQLite statements and parameter binding for values; schema/query fragments are static constants.
- Spanish user-visible messages and English identifiers/file names, with domain values in English.
- Component files use PascalCase; hooks use `use . . .`; services/modules use kebab or dot suffixes.

No consistent repository-wide formatting convention currently exists. Some source is highly compressed onto one line; other source uses conventional multiline formatting. There is no ESLint, Prettier, Stylelint, EditorConfig, or npm lint/format script.

Required invariant-preserving practices

When changing queue mutations, preserve transaction boundaries, server-side validation, active-number uniqueness, request-ID replay semantics, state broadcast, and the rule that only new/recall operations create announcements. Do not treat UI checks as authoritative validation.

When changing contracts, update both `server/models/state.ts` and `src/types.ts` until shared generation is introduced. When changing SPA paths, update both `App.tsx` and the static route list in `bootstrap.ts`.

Recommendations

Adopt one formatter and linter with checked-in configuration; add type-check/lint scripts and CI; share or generate API types; use stable domain error codes; format JSX and CSS for reviewability. These are recommendations, not current standards.

8.4 Git workflow

Observed repository history

The audit saw branches `main`, `copilot/comal-interactive-mockup`, and a local `codex/comal-interactive-mockup`; this documentation work starts from Copilot commit `bf71e73` on `docs/technical-documentation-v1`. There are no tags and no release branches. The inspected history is short and contains no evidence sufficient to establish a merge/rebase or release policy.

Most recent messages resemble Conventional Commits, especially `feat :`, `fix :`, and `docs :`, sometimes with scopes. Formatting is inconsistent (for example, extra spaces in some `feat :` entries). No commit-message hook or contributor guide enforces it.

Current workflow status

Branch naming is inconsistent by tool/purpose, and no protected-branch, review, merge, versioning, or tagging convention is documented in repository files. Therefore no workflow should be described as mandatory.

Recommended workflow

Use short-lived branches with a meaningful category/name, require review and passing build/test/audio checks before merging, prefer consistent Conventional Commit syntax, and tag releases only after a release/versioning

policy is approved. Define whether generated `dist`, `build-tests`, and binary audio belong in commits; current tracking/ignore state is inconsistent.

Evidence: Git refs and log inspected 14 September 2026; `.gitignore`

8.5 Configuration

Server defaults

Setting	Default	Source/behavior
Host	127.0.0.1	HOST; Electron forces loopback.
Port	3001	PORT; must be integer 1–65535; Electron forces 3001.
Database	data/comal.sqlite	DATABASE_PATH; Electron uses application user data.
Music root	data/music	MUSIC_PATH; directories created at startup.
Allowed origins	localhost/127.0.0.1 on 5173, 4173, and server port	Expanded by comma-separated ALLOWED_ORIGINS.

Persisted application defaults

If no configuration row exists, multimedia is fallback/stopped at volume 65 and unmuted. Settings use an 8-second visual announcement, automatic page rotation, and three Spanish footer messages. Writes merge partial input into current/default values and store the whole JSON document.

Frontend/build configuration

Vite development uses fixed port 5173 with strict-port behavior and proxies `/api`, `/media`, and `/socket.io`. The production build outputs `dist`. The server serves static content only when `dist/index.html` exists. TypeScript targets ES2022 and strict mode.

Electron packages Windows x64 NSIS, uses per-user installation by default, permits changing installation directory, includes `dist`, `build`, `electron`, package metadata, and copies `data/music` as extra resources.

Runtime configuration limits

Settings are global to the database, not per screen. Media playback progress/audio permission is browser-local. Environment variables are process settings only; there is no `.env` loader, configuration file parser, or UI for server settings.

Evidence: `server/main.ts`; `server/origins.ts`; `server/state.service.ts`; `vite.config.ts`; `package.json`; `electron/main.cjs`

8.6 Environment variables

Variable	Required	Component	Purpose/default
HOST	No	server	Listen address; defaults <code>127.0.0.1</code> . Electron sets it to loopback.
PORT	No	server/CORS	Listen port and allowed-origin port; defaults 3001. Must be integer 1–65535 in <code>main.ts</code> .
DATABASE_PATH	No	database	SQLite filename; defaults <code>data/comal.sqlite</code> . May be <code>:memory:</code> in direct construction/tests. Electron uses its user-data directory.
MUSIC_PATH	No	media	Root catalog directory; defaults <code>data/music</code> . Electron uses packaged extra resources.
ALLOWED_ORIGINS	No	HTTP/WebSocket CORS	Comma-separated additional origin strings; default empty in addition to built-in local origins. Values are trimmed but otherwise not validated.
ELECTRON_RUN_AS_NODE	Electron-managed	Electron child	Set to 1 so Electron's executable runs the compiled Node server.

No secret-valued environment variable is referenced. The repository ignores `.env` but does not load it automatically; environment provisioning belongs to the shell/process supervisor. Do not commit real data paths or future secrets.

Example for a LAN-bound standalone server (security review required):

```
$env:HOST = '0.0.0.0'
```

```
$env:PORT = '3001'  
$env:DATABASE_PATH = 'C:\comal-data\comal.sqlite'  
$env:MUSIC_PATH = 'C:\comal-data\music'  
npm start
```

Exposing the current unauthenticated API to a network is not a production recommendation; it merely illustrates actual variable use.

9 Operations

9.1 Deployment

Current deployment capability

Combined Node server. `npm run build` creates a Vite SPA and compiled Nest server. `npm start` serves the API, Socket.IO, media, and explicit SPA routes from one process. Default access is loopback. An external process manager/service definition is not provided.

Windows desktop. `electron-builder` is configured for an x64 NSIS installer and unpacked directory output. The app launches one cashier window, starts the embedded server, polls health for up to about 15 seconds, uses a per-user SQLite file, and terminates the child on app quit. The single-instance lock prevents a second app instance.

Development only. Vite/Nest split-process mode is not a production deployment.

Assets included

The Electron package includes the frontend, compiled backend, Electron entry point, package metadata, Node dependencies required by packaging, and `data/music`. Announcement/image assets are part of `dist`. Music is copied outside ASAR as an extra resource; runtime catalog mutation/write behavior in the installed resource location is not documented.

Absent deployment mechanisms

No Dockerfile, Compose, Nginx/reverse-proxy config, systemd/Windows service definition, cloud manifest, Kubernetes/Helm, CI/CD pipeline, code-signing config, auto-update, release channel, or rollback automation exists.

Topology decision required

For three physical displays described historically, stakeholders must choose either one mirrored graphics output or separate browser devices. The Electron server's forced loopback supports the former directly; the latter requires a secured LAN-bound server and validated CORS/firewall/startup behavior.

Evidence: `package.json`; `electron/main.cjs`; `server/bootstrap.ts`

9.2 Production considerations

The repository can produce runnable artifacts, but it does not establish production readiness.

Required before operational acceptance

1. Fix and verify public-display audio activation on every target display/browser.
2. Approve physical/network topology and decide whether API authentication or isolation is required.
3. Define/test SQLite backup, restore, retention, integrity check, and upgrade migration.
4. Validate target Windows version, hardware, displays, speakers, browser/Electron autoplay behavior, storage, and power-loss recovery.
5. Add repeatable frontend/E2E/package smoke checks and a release checklist.
6. Decide process supervision, startup, logs/rotation, health monitoring, incident fallback, and ownership.
7. Resolve licensing for application, dependencies, logos/images, synthesized audio, and distribution.

Capacity and reliability

No queue-size, client-count, latency, storage, uptime, or recovery target exists. Full snapshots contain all active turns and every catalog entry; history/requests grow in SQLite. Load testing is absent. Do not infer capacity from a successful two-client integration test.

Operational fallback

The prior README proposes printed tickets and verbal calls when local service, link, or power fails. That is a documented manual fallback, not an implemented recovery system, and requires stakeholder validation/training.

9.3 Monitoring

Implemented signals

- `GET /api/health` returns a static process-level OK response.
- Nest logs startup, warnings, and errors to `stdout/stderr`; `main.ts` logs the listen URL.
- Electron forwards child `stdout/stderr` to its console and reports startup failure in a dialog.
- Cashier/public interfaces expose current `Socket.IO` connection state.
- `MediaPlayer` exposes errors/notices in UI and puts playback status/time in data attributes, but nothing collects them.

Not implemented

No structured logging, file/event-log sink, log rotation, metrics, dashboards, alerting, tracing, audit trail, database size/integrity check, media inventory check at startup, synthetic transaction, or external health supervisor exists. The health route does not test SQLite or media access.

Recommended baseline

After defining the deployment topology, add a supervisor with restart policy, dependency-aware health/readiness, bounded structured logs, database-size/integrity and disk-space checks, and an operator-visible escalation procedure. Metrics and alerts need stakeholder-defined thresholds; none are prescribed here.

9.4 Backup and recovery

NOT_IMPLEMENTED as an operational procedure.

The durable state is a SQLite database plus operator-provided local music. In standalone mode the default database is `data/comal.sqlite`; Electron stores `comal.sqlite` in `app.getPath('userData')`, whose resolved directory varies by platform/user. SQLite WAL mode means copying only the main file while the app is writing may produce an inconsistent/incomplete backup.

Recovery facts

At normal restart, the database reopens, startup DDL ensures the current schema exists, and ready turns/configuration/request IDs are retained. Missing database creates an empty one. There is no corruption detection, automatic repair, backup discovery, restore UI/command, version compatibility check, or remote replica.

Procedure requiring approval

Stakeholders must define what to back up (database, WAL/SHM where applicable, music, installer/config), frequency, retention, destination, access protection, recovery point/time, acceptable downtime, and who tests restores. A safe implementation should use SQLite's online backup mechanism or a controlled shutdown/checkpoint rather than an ad hoc live file copy. This is a recommendation, not an existing feature.

Manual continuity

Printed tickets and verbal calls are the only repository-described contingency for service/power failure. The organization must validate and rehearse it.

Evidence: `server/database.ts`; `electron/main.cjs`; `.gitignore`; `historical README`

9.5 Security

Implemented controls

- CORS allow-list includes explicit local origins plus configured additions for HTTP and Socket.IO.
- Input validators reject unknown fields and constrain ticket numbers, counters, statuses, request IDs, settings, media primitives, playlist IDs, and YouTube providers.
- Server YouTube validation requires HTTPS, an exact allow-listed hostname, no credentials/port, and a valid video or playlist identifier.
- SQL values use prepared-statement parameters; dynamic SQL fragments are internal constants.
- Local media delivery resolves only entries discovered under the configured canonical root and stored in an allow-list map.

- Electron renderer enables context isolation and disables Node integration.
- Generated engine download is pinned and SHA-256 checked when the optional audio preparation mode is used.

Security gaps

- No authentication/authorization, CSRF protection, rate limiting, brute-force controls, or audit identity.
- State and history endpoints expose operational data to every reachable client; all mutations are equally open.
- Default transport is plain HTTP/WebSocket; no TLS or trusted reverse proxy is configured.
- ALLOWED_ORIGINS accepts arbitrary configured strings and CORS is not an authorization mechanism.
- No secret-management need exists today, but no framework is established for future secrets.
- SQLite/media are unencrypted at rest and filesystem permissions are left to the host.
- Dependency vulnerability scanning, SBOM, code signing, update provenance, and security response policy are absent.
- External YouTube use sends public-screen traffic to Google/YouTube and introduces privacy, availability, and content-policy considerations not documented by stakeholders.

Recommendations

First define the trust boundary: single locked-down PC with mirrored displays versus reachable LAN service. For LAN use, restrict bind/firewall exposure, authenticate mutation/admin operations, consider read-only display credentials or network segmentation, use TLS where appropriate, rate limit, log security-relevant changes, and protect backups. Run dependency/security review before release. Exact controls depend on stakeholder risk acceptance.

Evidence: `server/bootstrap.ts`; `server/origins.ts`; `server/dto/validation.ts`; `server/media/media.service.ts`; `electron/main.cjs`; `package.json`

10 Planning

10.1 Current engineering status

Baseline: commit bf71e73927fc7a24ec01501c38122327a64ffa4f, audited 14 September 2026.

Area	Classification	Evidence-based assessment
Project stage	Prototype / pre-production	Runnable local web and Electron implementation exists; operational acceptance/release policy does not.
Product definition	Partial	Ready-only workflow is clear; ownership, canonical brand, measurable outcomes, topology, and acceptance authority are undefined.
Frontend	IMPLEMENTED; gaps remain	Cashier/public pages and media/settings UI exist; no automated frontend tests, missing public empty state and audio activation.
Backend	IMPLEMENTED	Nest endpoints, validation, domain service, media service, and gateway function; architecture is single-module/local.
Database	IMPLEMENTED; gaps remain	SQLite schema, constraints, WAL, persistence work; migrations, retention, audit metadata, integrity/recovery absent.
Authentication	NOT_IMPLEMENTED	No identity, session, guard, role, or permission enforcement.
Queue management	IMPLEMENTED	Create ready, duplicate prevention, request replay, recall, counter update, deliver, cancel, reuse.
Client display	PARTIALLY_IMPLEMENTED	Ready list, paging, visual callout, footer, media/fallback and disconnect warning work; no explicit empty state or audio activation.
Multimedia	PARTIALLY_IMPLEMENTED	Local catalog and optional YouTube/fallback are implemented; sound is held muted by unreachable public activation and playback is unsynchronized.
Realtime	IMPLEMENTED	Full snapshots and ephemeral announcements with reconnect restoration; best-effort only and single-process.

Testing	Partial	Five backend/service/integration tests pass; type checks pass; frontend, E2E, accessibility, packaging, load, and recovery absent.
Deployment	Partial	Combined server and Windows NSIS build configured; no CI/CD, service supervision, signing, upgrade/rollback, or approved multi-screen topology.
Operations	NOT_IMPLEMENTED; unclear	Basic health/logs only; monitoring, backup/restore, retention, incident ownership, and procedures undefined.
Documentation	Rebuilt / validated	Structured LaTeX set reflects current source and uses the vendored Overleaf Software Manual template; the A4 PDF compiles and has been visually inspected.

Validation results

- `npm test`: 5 tests, 5 passed, 0 failed (validated 14 September 2026).
- Frontend TypeScript `--noEmit`: passed.
- Backend TypeScript `--noEmit`: passed.
- Production build: passed; Vite transformed 84 modules and backend compilation completed.
- Audio assets: passed; 102 announcement WAVs and six demo music tracks were structurally valid, PCM 16-bit, 22,050 Hz, mono, and non-silent.
- TeX compilation: `latexmk` with MiKTeX pdfTeX completed successfully; the resulting A4 PDF was rasterized and visually inspected page by page.

Critical gaps before production

Public audio activation, deployment trust/security decision, recovery procedure, frontend/system acceptance tests, target hardware validation, operational supervision, release/licensing decisions, and canonical product naming.

10.2 Product roadmap

This roadmap contains no dates or commitments. Items derive from gaps between documented ready-order intent and current behavior.

Priority	Capability	Evidence-based outcome
P0	Audible public calls	Provide an explicit, tested audio-activation journey per public screen; verify number/suffix sequencing and media duck/resume offline.
P0	Safe operating model	Decide whether displays are mirrored or network clients; approve security and manual-continuity model for that topology.
P0	Recoverable operation	Approve and exercise backup/restore and power/service failure procedures.
P1	Operational acceptance	Validate turn creation, recall, delivery, cancellation, restart, and disconnect workflows on target hardware and screens.
P1	Display completeness	Add/approve an empty-queue message and validate page rotation/viewing distance/readability.
P1	Administrative boundary	Decide whether media/settings require a distinct administrator role or remain cashier functions.
P1	History policy	Define who may view history, required fields, retention, search/export, and privacy treatment.
P2	Operator efficiency	Validate need for search/filter, scanner/keyboard flow, error correction, and long-queue handling before implementing them.
P2	Media policy	Define allowed content, Internet/privacy expectations, per-screen sync need, and local library ownership.
P3	Brand/visual evolution	Select canonical COMAL/Comal++/Troyanos identity and decide fate of unused mascot/reference assets.

Preparation tracking, POS integration, payments, inventory, invoicing, and ticket printing remain explicitly outside the prior documented scope; they are not roadmap items without a new stakeholder decision.

10.3 Technical roadmap

Priority	Workstream	Completion evidence
P0	Wire public audio activation	User gesture reaches <code>enableAnnouncementAudio</code> ; browser test proves sequential calls and media duck/resume.

P0	Security/topology design	Threat model and bind/firewall/auth decision approved; tests match chosen boundary.
P0	SQLite recovery	WAL-safe backup/restore documented, automated where appropriate, and tested with acceptance criteria.
P1	Continuous verification	CI runs clean install, type checks, backend tests, audio verifier, and production build.
P1	Frontend/E2E coverage	Automated two-page lifecycle, reconnect, empty state, settings, media fallback, and accessibility smoke tests.
P1	Schema lifecycle	Versioned migrations, upgrade/rollback policy, closure audit fields if approved, and integrity checks.
P1	Retention	Stakeholder-approved turn/request retention and safe pruning/archive implementation.
P1	Production runtime	Supervision, startup, logs/rotation, dependency-aware health, disk/database monitoring, installer smoke/signing decision.
P2	Contract alignment	Shared/generated schemas correct timestamps/settings types and unify browser/server validation.
P2	Error model	Stable domain error codes, frontend error boundary, and actionable recovery messages.
P2	Code quality	Formatter/linter and readable JSX/CSS enforced; tracked generated artifacts policy resolved.
P3	Modularization	Split state/media/config modules only when testing/change isolation justifies it.

No scalability redesign is proposed until screen count, queue size, and latency targets are measured and approved.

10.4 Release plan

No release tags, versioning policy, target date, environment promotion flow, or release owner exists. Package version is 0.2.0, but the repository does not define what that version means.

Proposed readiness gates (not yet adopted)

1. **Scope approval:** owner confirms ready-only workflow, brand, users, topology, history/retention, security, hardware, and acceptance authority.

2. **P0 implementation:** public audio activation and recovery/security requirements are satisfied.
3. **Automated quality:** clean install, type checks, backend tests, frontend/E2E checks, audio asset verifier, and production build pass in a repeatable environment.
4. **Target acceptance:** installer/server is exercised on actual cashier/display/audio hardware, offline and after restart; manual fallback is rehearsed.
5. **Operational handoff:** signed-off backup/restore, logs/monitoring, support contacts, data location, update/rollback, and content ownership.
6. **Release record:** approved version/tag, artifact checksum/provenance, known limitations, rollback instructions, and license decision.

Current disposition

The repository is not demonstrably production-ready because P0 audio/recovery/security-topology decisions and repeatable frontend/package acceptance remain open. This assessment does not assign a percentage or delivery date.

10.5 Project metrics

Observed engineering inventory

At the audit baseline:

- 9 JSON API routes plus 1 local media-file route;
- 2 Socket.IO application events (state, announcement);
- 3 SQLite tables and 2 explicit indexes;
- 3 turn statuses and 3 counter values;
- 2 page components and 3 cashier workspace sections;
- 5 automated backend/service/integration tests, all passing during the audit;
- 99 numbered announcement WAVs, 2 counter WAVs, 1 no-counter suffix, and 6 generated demo music tracks;
- 0 frontend/component/E2E tests, 0 CI workflows, 0 release tags, and 0 repository license files.

Counts describe repository contents, not quality, progress, coverage, throughput, adoption, or production readiness.

Undefined product/operational metrics

No telemetry or requirements define average wait, pickup time, abandoned turns, daily volume, queue length, announcement success, display connectivity, error rate, uptime, recovery time, or user satisfaction. Adding metrics may require privacy/retention decisions and should follow stakeholder goals rather than collecting data speculatively.

Recommended engineering indicators

After CI/operations exist, useful evidence could include build/test result, restore-drill result/date, database size/integrity, display connection health, and failed mutation/media counts. Thresholds and retention are Requires stakeholder validation.

11 Reference

11.1 Terminology and naming rules

Use these names consistently in technical work:

Canonical documentation term	Source/UI aliases	Rule
COMAL Queue	Comal++, comal-local, Troyanos	Use COMAL Queue for the system until stakeholders choose final product branding; quote UI/package names when exact.
Turn	turno, ready order, ticket record	Use “turn” for the stored queue entity; “ticket number” only for visible number.
Ready	listo, listo para recoger	Only active status; do not call it waiting/preparing.
Delivered	entregado	Terminal stored status; not deleted.
Cancelled	retirado, capture error	Terminal stored status; avoid “deleted” unless discussing nonexistent physical deletion.
Recall	volver a anunciar	Same ready entity, new rank/timestamp/announcement.
Cashier workspace	panel de caja, admin workspace	Current combined operational/configuration page; not an authenticated admin console.
Public display	pantalla pública, client display	Read-only React page at /pantalla.
Counter	mostrador, pickup point	Domain values 0/1/2; 0 means no assigned counter.
Announcement	llamado, call	Ephemeral live event/overlay; not a turn status or table.
Local service	servidor local	NestJS/Socket.IO process serving API and state.

The Spanish UI replaces configured occurrences of “Comal++” in footer messages with “Troyanos” at render time. This behavior is presentation-specific and should not be mistaken for data migration.

11.2 Repository map

```
comal-queue-system/
|-- README.md           concise Markdown entry point
|-- docs/               LaTeX product/process source of
truth
|   |-- technical-documentation.tex
|   |-- preamble.tex
|   |-- 01-product/ ... 11-reference/
|-- src/
|   |-- pages/          cashier and public composition
|   |-- components/     queue/media/settings/display UI
|   |-- hooks/          socket state, announcements,
clock
|   |-- services/       HTTP, YouTube, announcement audio
|   |-- styles/         base/admin/public CSS
|-- server/
|   |-- app.controller.ts HTTP endpoints
|   |-- state.service.ts queue/config domain
|   |-- database.ts      SQLite/schema
|   |-- events.gateway.ts Socket.IO
|   |-- dto/validation.ts input validation
|   |-- media/media.service.ts local catalog and allow-list
|-- tests/backend.test.ts all automated tests
|-- electron/main.cjs    desktop shell/server lifecycle
|-- scripts/dev.mjs      dev process orchestration
|-- tooling/audio/       asset generation/verification
|-- public/              images, logos, announcement WAVs
|-- data/music/          local/demo music
|-- package.json         commands/dependencies/packaging
|-- vite.config.ts       frontend build/dev proxies
|-- tsconfig*.json       frontend/backend TypeScript
```

Feature-to-code index

Question	Start here
How is a turn created/closed?	server/state.service.ts; server/dto/validation.ts; src/pages/CashierPage.tsx; ReadyList.tsx
How is state synchronized?	server/events.gateway.ts; src/hooks/useSystem.tsx

How are calls queued/played?	src/hooks/useAnnouncements.ts; src/services/announcementAudio.ts
How is media discovered/served?	server/media/media.service.ts; MediaController
How is media rendered?	src/components/MediaPlayer.tsx; src/services/youtube.ts
Where is the database model?	startup DDL in server/database.ts; types in server/models/state.ts
What is tested?	tests/backend.test.ts; tooling/audio/verify-audio.mjs
How is Windows packaged?	package.json build block; electron/main.cjs

11.3 Documentation status and confidence

Area	Status	Evidence quality	Stakeholder input needed
Product vision	Partial	Medium	Yes: brand, owner, goals
Business context	Partial	Medium	Yes: hardware, topology, and operation
Current scope	Documented	High	Yes for acceptance
Out-of-scope boundary	Documented historical intent	Medium	Yes if scope changes
User groups	Documented as UI roles	High	Yes for real responsibilities
Personas/research	Partial	Low	Yes; no research artifacts
Queue lifecycle	Documented	High	No for current code
Database/schema	Documented	High	Yes for retention/evolution
HTTP API	Documented	High	Yes for stability/versioning
Realtime events	Documented	High	Yes for delivery expectations
Multimedia	Documented with gaps	High	Yes for policy/sync
Public audio	Gap documented	High	Yes for acceptance/hardware

Authentication/roles	Not implemented	High	Yes for trust model
UX behavior	Documented	High	Yes for formal usability/a11y
Testing	Documented	High	Yes for required coverage
Security	Partial assessment	High code evidence	Yes for threat/risk acceptance
Deployment	Partial	High	Yes for target topology/signing
Backup/recovery	TBD	High absence evidence	Yes: RPO/RT0/owner
Monitoring/support	TBD	High absence evidence	Yes: targets/ownership
Roadmaps	Recommended, not committed	Medium	Yes: priority/commitment
Project metrics	Inventory only	High	Yes for product/SLO metrics
License	Not documented	High absence evidence	Yes

Validation record

The documentation was checked against all tracked TypeScript/TSX source, package/build configuration, tests, current Markdown documents, audio tooling, assets/catalog metadata, environment-variable references, routes, domain terms, and recent Git history. Source paths cited in documents existed at the audit baseline. The backend suite, frontend/backend TypeScript checks, production build, and audio asset verifier passed.

LaTeX compilation is VALIDATED. The combined document builds with the vendored `ol-softwaremanual` class, A4 geometry, table of contents, and code-listing index. A static pass confirmed that all 58 master `\input` targets exist, only the root README remains Markdown, environment delimiters are paired, and raw brace counts are balanced across 60 TeX files. The resulting PDF was rasterized and visually inspected page by page.

Known superseded documentation

The previous root README claimed the repository contained no executable application; source contradicts it. `docs/UI-REDESIGN.md` and `tooling/`

audio/README.md were removed after verified content was migrated into this hierarchy. Current code, tests, and this status matrix supersede those claims.